

Lesson 27 Notes:

1. What is Bokeh?

- **Bokeh** is a Python library for **interactive visualizations in modern web browsers**.
 - Lets you create rich, JavaScript-powered plots **without writing JavaScript**.
 - Can produce:
 - Standalone **HTML** files.
 - **Notebook**embedded plots.
 - Interactive apps via **Bokeh server**.
-

2. Installation

Install with **pip**:

```
pip install bokeh
```

or with **conda**:

```
conda install bokeh
```

3. Running Bokeh in Jupyter Notebooks

To display plots directly inside a notebook:

```
from bokeh.io import output_notebook

output_notebook()
```

- Call `output_notebook()` **once per notebook** to initialize inline output.

- Use `show(...)` to render each plot cell.
-

4. How Bokeh Works (Components)

- Python side creates a **Bokeh document** (plots, widgets, layouts, etc.).
 - This is converted to **JSON**.
 - **BokehJS** (a JavaScript library) in the browser reads that JSON and renders the visualization.
 - You usually stay on the Python side; BokehJS runs automatically in the browser.
-

5. Two Main Interfaces

1. High-level `bokeh.plotting`

- Main, user-friendly interface.
- Similar to Matplotlib's plotting API.
- Example functions: `figure` , `.line()` , `.circle()` , `.scatter()` , `show()` .

2. Low-level `bokeh.models`

- Full control of every component (axes, tools, glyphs, widgets).
- More verbose but highly customizable.

Most of the time you start with `bokeh.plotting` and drop into `bokeh.models` when you need extra control.

6. Building Blocks & Tools

Documents & Plots

- A **document** = full Bokeh visualization (can contain multiple plots and widgets).
- A **plot** is the main drawing area:
 - Contains **glyphs** (visual marks),

- **Axes, grids, legends, annotations,**
- A **toolbar** with interactive tools.

Toolbar Tools (common ones)

- **Pan** – drag to move view.
- **Box Zoom** – drag rectangle to zoom.
- **Wheel Zoom** – zoom with mouse wheel.
- **Save** – export as PNG.
- **Reset** – return to original view.
- **Help** – info on tools.

7. Glyphs (Visual Marks)

- A **glyph** is how your data appears: line, circle, square, bar, wedge, etc.
- You pass your data to glyph methods on a `figure` object, e.g.:

```
from bokeh.plotting import figure, show
```

```
p = figure(title="Simple Line")  
x = [1, 2, 3, 4, 5]  
y = [2, 5, 8, 2, 7]
```

```
p.line(x, y, line_width=2)
```

```
show(p)
```

Common glyph methods on a figure: `.line()`, `.circle()`, `.square()`, `.scatter()`, etc.

8. Multiple Glyphs & Legend Interactivity

You can layer several glyphs on the same plot and make legends interactive.

```

from bokeh.plotting import figure, show

x_vals = [1, 2, 3, 4, 5]

p = figure(title="Multiple Glyphs Example")

# Line glyph
p.line(x_vals, [2, 5, 8, 2, 7],
       line_width=2, legend_label="Line")

# Circle markers
p.circle(x_vals, [2, 5, 8, 2, 7],
         size=10, color="red", legend_label="Circles")

# Square markers with different y-values
p.square(x_vals, [1, 4, 7, 1, 6],
         size=12, color="navy", alpha=0.5, legend_label="Squares")

# Make legend items clickable (hide/show glyphs)
p.legend.click_policy = "hide"

show(p)

```

Key points:

- Multiple glyph calls on the same `figure` overlay graphics.
- `legend_label` creates legend entries automatically.
- `legend.click_policy = "hide"` allows toggling visibility.

9. Widgets (Interactivity Controls)

Bokeh supports widgets like sliders, dropdowns, and date pickers.

Example: **DateRangeSlider** from `bokeh.models`:

```
from bokeh.models import DateRangeSlider
from bokeh.io import show
```

```
date_range_slider = DateRangeSlider(
    value=("2022-10-01", "2022-12-31"),
    start="2022-07-01",
    end="2023-03-31",
)
```

```
show(date_range_slider)
```

- Widgets can be combined with callbacks (Python or JavaScript) to control plots.
- In more advanced apps, these widgets are linked to data and plots via the Bokeh server.

10. Layouts (Combining Multiple Plots)

A Bokeh document can contain multiple plots and widgets arranged in a **layout**.

Example: row layout with three scatter plots

```
from bokeh.layouts import row
from bokeh.plotting import figure, show
```

```
x = list(range(11))
y0 = x
y1 = [10 - i for i in x]
y2 = [abs(i - 5) for i in x]
```

```
s1 = figure(width=250, height=250, background_fill_color="#fafafa")
s1.scatter(x, y0, size=12, color="#53777a", alpha=0.8, marker="circle")
```

```
s2 = figure(width=250, height=250, background_fill_color="#fafafa")
s2.scatter(x, y1, size=12, color="#c02942", alpha=0.8, marker="triangle")
```

```
s3 = figure(width=250, height=250, background_fill_color="#fafafa")
s3.scatter(x, y2, size=12, color="#d95b43", alpha=0.8, marker="square")

show(row(s1, s2, s3))
```

Example: two functions side-by-side (quadratic vs cubic)

```
from bokeh.plotting import figure, show
from bokeh.layouts import row
import numpy as np

x = np.linspace(0, 10, 100)
y_quad = x**2
y_cube = x**3

p1 = figure(width=300, height=300, title="Quadratic (x^2)")
p1.line(x, y_quad, color="blue", line_width=3)

p2 = figure(width=300, height=300, title="Cubic (x^3)")
p2.line(x, y_cube, color="green", line_width=3)

show(row(p1, p2))
```

- `row(...)` places elements horizontally.
- There is also `column(...)` and more complex layout options (e.g. `gridplot`).